

Project: Processi di Hawkes

eventi auto-eccitanti, memoria stocastica e clustering temporale

1. Obiettivi della dispensa

Questa dispensa introduce i processi di Hawkes come caso di studio per un corso di metodi computazionali per modelli stocastici.

Gli obiettivi sono sei:

1. formalizzare l'idea di evento auto-eccitante tramite il concetto di intensita' condizionata;
2. derivare le proprieta' fondamentali del processo di Hawkes univariato: intensita' media stazionaria, condizione di stabilita', funzione di autocorrelazione;
3. mostrare come simulare il processo con l'algoritmo di Ogata (thinning);
4. introdurre il caso multivariato e discutere le applicazioni alle reti di influenza;
5. collegare il processo di Hawkes alla stima della massima verosimiglianza e al problema dell'inferenza;
6. discutere i limiti del modello e le sue estensioni naturali.

Dal punto di vista del corso, i processi di Hawkes sono particolarmente interessanti perche' introducono un ingrediente nuovo rispetto ai modelli gia' visti: la **dipendenza dalla storia passata**. L'intensita' con cui avvengono nuovi eventi non e' costante nel tempo, ma cresce ogni volta che un evento si verifica, e poi decade. Questo produce clustering temporale — gli eventi tendono a venire in grappoli — un fenomeno osservato in contesti molto diversi.

2. Motivazione: cosa descrive un processo di Hawkes

Un processo di Hawkes modella sequenze di eventi in cui ogni evento aumenta temporaneamente la probabilita' che si verifichino altri eventi.

Esempi concreti e molto diversi tra loro:

- **Sismologia.** Un terremoto principale (mainshock) e' seguito da scosse di assestamento (aftershocks) che a loro volta possono generare altre scosse. Il modello ETAS (Epidemic Type Aftershock Sequence), fondamento della sismologia moderna, e' un processo di Hawkes multivariato. La sequenza di aftershocks del terremoto del 2009 in Abruzzo o del 2016 in Centro Italia seguono esattamente questa struttura.
- **Criminalita'.** Un crimine in una zona aumenta la probabilita' di crimini successivi nella stessa zona nelle ore o nei giorni seguenti (near-repeat victimization). Le forze dell'ordine usano modelli di Hawkes per prevedere dove si concentreranno i reati e ottimizzare le pattuglie.
- **Finanza.** Le transazioni ad alta frequenza nei mercati finanziari si aggregano in cluster: un ordine grande scatena una cascata di ordini di risposta. I flash crash — crolli istantanei del mercato — possono essere letti come eventi esplosivi in un processo di Hawkes instabile.
- **Social media.** Un post che diventa virale genera retweet, che generano altri retweet. La diffusione di una notizia su Twitter o la propagazione di un meme seguono una dinamica auto-eccitante molto simile a quella di Hawkes.
- **Neuroscienze.** Le scariche di un neurone aumentano temporaneamente la probabilita' che lo stesso neurone scarichi di nuovo (refrattarieta' eccitante) e che i neuroni connessi scarichino (propagazione del segnale). Reti di neuroni sono naturalmente modellate con processi di Hawkes multivariati.
- **Epidemiologia.** Ogni caso di una malattia infettiva genera nuovi contagi con un ritardo variabile. Il numero di riproduzione di base R_0 in epidemiologia corrisponde esattamente al parametro di branching di un processo di Hawkes.

In tutti questi casi, la struttura e' la stessa: un evento accade, aumenta temporaneamente il rischio di altri eventi, che a loro volta aumentano il rischio, e cosi' via. Il sistema ha **memoria**: il passato recente conta.

3. Il processo di Poisson come punto di partenza

3.1 Processo di Poisson omogeneo

Il processo di Poisson omogeneo con tasso λ e' il modello piu' semplice per una sequenza di eventi casuali. Gli eventi arrivano a un ritmo costante: il numero di eventi in un intervallo $(s, t]$ e' una variabile di Poisson con media $\lambda(t - s)$, e gli eventi in intervalli disgiunti sono indipendenti.

La caratteristica fondamentale e' che il processo non ha memoria: la probabilita' che avvenga un evento nel prossimo istante non dipende da quando e' avvenuto l'ultimo evento.

Esempio concreto. Le telefonate a un call center nelle ore di punta arrivano con un tasso approssimativamente costante. Sapere che la ultima chiamata e' arrivata un secondo fa o un minuto fa non cambia la probabilita' della prossima.

3.2 Processo di Poisson non omogeneo

Un passo verso la dipendenza dal passato e' il processo di Poisson non omogeneo, in cui il tasso $\lambda(t)$ varia nel tempo in modo deterministico.

Esempio concreto. Le visite a un sito web sono piu' frequenti la mattina e il pomeriggio che di notte. Il tasso $\lambda(t)$ segue una curva periodica.

Il processo di Hawkes va oltre: il tasso varia in modo stocastico, dipendendo dalla storia degli eventi passati.

4. Definizione del processo di Hawkes

4.1 L'intensita' condizionata

Sia $\{t_k\}_{k \geq 1}$ la sequenza degli eventi in ordine cronologico, con $t_1 < t_2 < t_3 < \dots$

Sia $\mathcal{H}_t = \{t_k : t_k < t\}$ la storia del processo fino al tempo t (escluso).

Il processo di Hawkes univariato e' definito dalla sua **intensita' condizionata**:

$$\lambda^*(t) = \mu + \sum_{t_k < t} \phi(t - t_k).$$

I tre ingredienti sono:

- $\mu > 0$ e' il **tasso di background** (o tasso esogeno): e' la probabilita' di base che avvenga un evento, in assenza di qualsiasi storia passata. Modella gli eventi che arrivano dall'esterno del sistema.
- $\phi(s) \geq 0$ e' il **kernel di eccitazione**: descrive quanto un evento al tempo t_k aumenta la probabilita' di nuovi eventi un tempo $s = t - t_k$ dopo. Il kernel decade nel tempo: l'effetto di ogni evento si esaurisce progressivamente.
- La somma scorre su tutti gli eventi passati: ogni evento contribuisce alla probabilita' corrente di un nuovo evento.

4.2 Interpretazione

L'intensita' $\lambda^*(t)$ e' la probabilita' condizionata che un evento avvenga nell'intervallo $(t, t + dt]$ dato tutto quello che e' successo prima:

$$P(\text{evento in } (t, t + dt] \mid \mathcal{H}_t) = \lambda^*(t) dt + o(dt).$$

Quando avviene un evento, l'intensita' salta verso l'alto di un ammontare $\phi(0)$, poi decade seguendo la forma del kernel. Se nel frattempo arrivano altri eventi, l'intensita' sale di nuovo.

L'intensita' e' quindi una somma di impulsi decrescenti, uno per ogni evento passato, piu' il background. Questa struttura a "impulsi sovrapposti" e' il cuore del modello.

4.3 Kernel esponenziale

La scelta piu' comune per il kernel e' quella esponenziale:

$$\phi(s) = \alpha e^{-\beta s}, \quad s \geq 0,$$

con $\alpha > 0$ e $\beta > 0$.

- α e' l'**ampiezza** dell'eccitazione: quanto ogni evento aumenta immediatamente l'intensita';
- β e' il **tasso di decadimento**: quanto velocemente l'effetto si esaurisce; la vita media dell'eccitazione e' $1/\beta$.

Con il kernel esponenziale, l'intensita' condizionata soddisfa una semplice equazione differenziale tra un evento e il successivo:

$$\lambda^*(t) = \mu + \sum_{t_k < t} \alpha e^{-\beta(t-t_k)}.$$

Questa forma e' particolarmente comoda perche' l'intensita' si aggiorna in modo ricorsivo: quando avviene un nuovo evento al tempo t_n , basta aggiungere α all'intensita' corrente.

Esempio concreto. Dopo un terremoto di magnitudo 6, le scosse di assestamento seguono il decadimento di Omori: la frequenza degli aftershocks decade come una legge di potenza nel tempo. Il kernel esponenziale approssima questo decadimento nella versione base del modello; le versioni avanzate usano kernel a legge di potenza.

4.4 Il branching ratio

L'integrale del kernel e' il parametro piu' importante del modello:

$$n = \int_0^\infty \phi(s) ds.$$

Per il kernel esponenziale $\phi(s) = \alpha e^{-\beta s}$:

$$n = \frac{\alpha}{\beta}.$$

Il parametro n e' chiamato **branching ratio** ed ha un'interpretazione molto precisa: e' il numero medio di eventi di "seconda generazione" generati da ciascun evento. Ogni evento genera in media n discendenti diretti, ognuno dei quali ne genera altri n , e cosi' via.

La condizione di stabilita' del processo e':

$$n < 1.$$

Interpretazione. Se $n < 1$, ogni "cascata" di eventi si esaurisce in media: il processo e' stazionario. Se $n \geq 1$, le cascate possono diventare infinite e il processo esplode. Questa condizione e' esattamente analoga al numero di riproduzione $R_0 < 1$ in epidemiologia, o al fattore di ramificazione di un processo di Galton-Watson.

Esempio concreto. Nei mercati finanziari, stime empiriche del branching ratio per le transazioni ad alta frequenza danno valori di n molto vicini a 1, specialmente durante i periodi di alta volatilita'. Questo spiega la vulnerabilita' dei mercati ai flash crash: il sistema e' vicino all'instabilita'.

5. Proprieta' stazionarie

5.1 Intensita' media stazionaria

Per $n < 1$, il processo raggiunge uno stato stazionario. L'intensita' media e':

$$\bar{\lambda} = \frac{\mu}{1 - n}.$$

Il fattore $1/(1 - n)$ e' il moltiplicatore del processo di branching: ogni evento esogeno (con tasso μ) genera in media $1/(1 - n)$ eventi totali (incluse tutte le generazioni di discendenti).

Esempio concreto. Un sito di notizie riceve in media $\mu = 10$ commenti originali all'ora. Ogni commento genera in media $n = 0.5$ commenti di risposta. Il numero totale di commenti all'ora e' $10/(1 - 0.5) = 20$: meta' sono originali, meta' sono risposte a risposte a risposte...

5.2 Funzione di autocorrelazione

Il processo di Hawkes mostra clustering temporale: gli eventi non sono distribuiti uniformemente nel tempo, ma tendono a venire in grappoli. Questo si misura con la funzione di autocorrelazione del processo puntuale.

Per il kernel esponenziale, la funzione di autocorrelazione decade esponenzialmente con un tasso che dipende sia da β (il decadimento del kernel) sia dalla struttura del branching.

La presenza di autocorrelazione positiva e' la firma del clustering: vedere molti eventi recentemente aumenta la probabilita' di vedere eventi nel prossimo futuro.

5.3 Struttura di branching

Il processo di Hawkes ammette una rappresentazione come processo di branching: gli eventi si dividono in due categorie.

Immigranti (o eventi esogeni): arrivano secondo un processo di Poisson con tasso μ . Sono gli eventi "originali", non causati da eventi precedenti.

Discendenti: ogni evento genera una progenie di discendenti secondo un processo di Poisson con tasso $\phi(s)$ un tempo s dopo. I discendenti generano a loro volta altri discendenti.

Il processo totale e' la sovrapposizione di tutti questi alberi di discendenza. Questa struttura e' molto utile sia per l'interpretazione che per la simulazione.

Esempio concreto. Su Twitter, alcuni tweet sono originali (immigranti), altri sono retweet di retweet (discendenti). Il branching ratio misura quanto si propaga mediamente un tweet originale.

6. Il processo di Hawkes multivariato

6.1 Struttura

Nel caso multivariato, ci sono M tipi di eventi distinti, ciascuno con la propria intensita' condizionata. L'intensita' del processo di tipo m e':

$$\lambda_m^*(t) = \mu_m + \sum_{j=1}^M \sum_{t_k^{(j)} < t} \phi_{mj}(t - t_k^{(j)}),$$

dove $\phi_{mj}(s)$ descrive quanto un evento di tipo j eccita il processo di tipo m dopo un ritardo s .

La matrice (ϕ_{mj}) e' la **matrice di influenza**: descrive la struttura delle eccitazioni incrociate tra i diversi tipi di eventi.

6.2 Stabilita' multivariata

La condizione di stabilita' si generalizza: la matrice dei branching ratio

$$N_{mj} = \int_0^\infty \phi_{mj}(s) ds$$

deve avere raggio spettrale (massimo autovalore in modulo) strettamente minore di 1.

6.3 Esempi applicativi

Sismologia. I tipi di evento sono le diverse zone geografiche. Un terremoto in una zona eccita sia la stessa zona (aftershocks) sia le zone vicine (stress transfer). La matrice di influenza codifica la struttura spaziale delle interazioni sismiche.

Criminalita'. I tipi sono diverse categorie di reato (rapine, furti, aggressioni) in diverse zone. Un reato in una zona eccita sia la stessa categoria di reato nella stessa zona sia categorie diverse (ad esempio, una rapina aumenta la probabilita' di furti nelle vicinanze).

Neuroscienze. I tipi sono i diversi neuroni in una rete. La matrice di influenza e' la matrice di connettivita' della rete neurale. Stimare questa matrice dai dati di attivita' neurale e' uno dei problemi centrali della neuroscienza computazionale.

Finanza. I tipi sono diversi asset o diversi tipi di ordine (acquisto/vendita). La matrice di influenza descrive come gli ordini su un asset eccitano gli ordini su altri asset.

7. Simulazione: l'algoritmo di Ogata

7.1 Idea del thinning

L'algoritmo di Ogata (1981) e' il metodo standard per simulare un processo di Hawkes. Si basa sul **thinning**, una tecnica generale per simulare processi di Poisson non omogenei.

L'idea e': un processo di Poisson non omogeneo con intensita' $\lambda^*(t)$ puo' essere simulato partendo da un processo di Poisson omogeneo con tasso costante $\Lambda \geq \lambda^*(t)$ per tutto il periodo e poi scartando ("assottigliando") gli eventi in eccesso con la probabilita' giusta.

7.2 Algoritmo

Per un processo di Hawkes con kernel esponenziale, l'algoritmo e' il seguente.

Sia t il tempo corrente e λ^* l'intensita' condizionata corrente.

1. Trova un **upper bound** $\Lambda \geq \lambda^*(s)$ per tutti i $s \geq t$ nel prossimo intervallo. Per il kernel esponenziale, l'intensita' e' decrescente tra un evento e il successivo, quindi $\Lambda = \lambda^*(t)$ e' un upper bound valido fino al prossimo evento.

2. Genera il **candidato**: $\Delta t \sim \text{Exp}(\Lambda)$, poni $t' = t + \Delta t$.
3. **Accetta o rifiuta**: calcola l'intensita' effettiva $\lambda^*(t')$. Accetta t' come evento reale con probabilita' $\lambda^*(t')/\Lambda$. Altrimenti, rifiuta e ripeti dal punto 1 con il nuovo tempo t' .
4. Se l'evento e' accettato al tempo t' : aggiorna l'intensita' (aggiungi α all'intensita' corrente, poi decadi), registra t' come evento, e torna al punto 1.

7.3 Perché funziona

Il thinning produce eventi distribuiti esattamente secondo l'intensita' condizionata $\lambda^*(t)$. La dimostrazione si basa sul fatto che il processo di Poisson omogeneo di tasso Λ genera “troppi” eventi, e il rifiuto di una frazione $1 - \lambda^*(t)/\Lambda$ di essi corregge esattamente la distribuzione.

7.4 Efficienza

Per il kernel esponenziale l'algoritmo e' molto efficiente: l'upper bound $\Lambda = \lambda^*(t)$ e' molto vicino all'intensita' reale, quindi pochi eventi vengono rifiutati. Per kernel a legge di potenza l'algoritmo e' meno efficiente perche' l'intensita' decade lentamente.

8. Inferenza: stimare i parametri dai dati

Una delle applicazioni piu' importanti del modello e' l'inferenza: dati una sequenza di eventi osservati $\{t_1, \dots, t_n\}$, stimare i parametri (μ, α, β) .

8.1 Log-verosimiglianza

Per un processo puntuale osservato su $[0, T]$, la log-verosimiglianza e':

$$\log L(\mu, \alpha, \beta) = \sum_{k=1}^n \log \lambda^*(t_k) - \int_0^T \lambda^*(t) dt.$$

Il primo termine e' la somma dei logaritmi dell'intensita' nei momenti degli eventi: deve essere grande, cioe' il modello deve assegnare alta intensita' nei momenti in cui gli eventi accadono.

Il secondo termine e' l'integrale dell'intensita': deve essere piccolo, cioe' il modello non deve prevedere troppi eventi nei momenti in cui non accadono.

Per il kernel esponenziale, sia la somma sia l'integrale si calcolano in modo ricorsivo in $O(n)$ operazioni, rendendo la stima efficiente anche per sequenze lunghe.

8.2 Residui del processo

Per verificare la bonta' del modello si usa il **teorema di rescaling** (Papangelou, 1972): se il modello e' corretto, i tempi trasformati

$$\tau_k = \int_0^{\tau_k} \lambda^*(s) ds$$

formano un processo di Poisson omogeneo con tasso 1 sull'intervallo $[0, \Lambda(T)]$.

Quindi, i tempi inter-evento trasformati $\tau_k - \tau_{k-1}$ devono essere distribuiti esponenzialmente con media 1. Un test di bonta' del modello consiste nel verificare questa proprieta' con un QQ-plot o un test di Kolmogorov-Smirnov.

Questo e' uno degli strumenti diagnostici piu' potenti per valutare se un processo di Hawkes e' adeguato per un dato insieme di dati.

9. Limiti del modello e estensioni

9.1 Kernel esponenziale vs legge di potenza

Il kernel esponenziale e' comodo matematicamente, ma in molte applicazioni reali il decadimento e' piu' lento, seguendo una legge di potenza:

$$\phi(s) = \frac{\alpha}{(1 + s/\beta)^{1+\delta}}, \quad \delta > 0.$$

Questo e' il kernel del modello ETAS in sismologia (legge di Omori). La coda pesante significa che un evento puo' influenzare il sistema per molto tempo.

9.2 Eccitazione e inibizione

Il modello base di Hawkes ammette solo eccitazione ($\phi \geq 0$). In molte applicazioni, alcuni eventi riducono la probabilita' di eventi successivi (ad esempio, dopo un grosso ordine di acquisto in borsa, la probabilita' di altri acquisti immediati puo' diminuire). Modelli con kernel a segno variabile generalizzano la struttura.

9.3 Hawkes non lineare

Nel modello base, la dipendenza dall'intensita' e' lineare. Versioni non lineari sostituiscono l'intensita' condizionata con una funzione non lineare della storia passata, ad esempio:

$$\lambda^*(t) = \Phi \left(\mu + \sum_{t_k < t} \phi(t - t_k) \right),$$

dove Φ e' una funzione non negativa. Questo permette di modellare saturazione e inibizione.

9.4 Hawkes neurale

Versioni molto recenti sostituiscono il kernel parametrico con una rete neurale che impara la forma del kernel direttamente dai dati. Queste varianti sono molto flessibili ma richiedono grandi quantita' di dati per la stima.

10. Domande scientifiche che il modello permette di studiare

Il modello e' utile per affrontare domande molto concrete.

1. Come cambia il clustering degli eventi al variare del branching ratio n ?
2. Cosa succede quando $n \rightarrow 1$? Il processo mostra comportamenti critici?
3. Come si distingue un processo di Hawkes da un processo di Poisson non omogeneo sui dati?
4. Quanto conta la forma del kernel nel determinare le proprieta' stazionarie?
5. In un processo multivariato, e' possibile identificare la struttura di influenza dalla sola osservazione degli eventi?
6. Come cambia il comportamento del processo se il kernel ha coda pesante invece di decadimento esponenziale?

11. Schema del laboratorio

11.1 Laboratorio 1 - Simulazione e visualizzazione

Obiettivo

Simulare un processo di Hawkes univariato e osservare il clustering degli eventi.

Attivita'

1. fissare $\mu = 0.5$, $\alpha = 0.8$, $\beta = 1.0$ (branching ratio $n = 0.8$);
2. simulare il processo su $[0, T]$ con $T = 200$;
3. rappresentare gli eventi su una linea temporale;
4. stimare empiricamente l'intensita' media e confrontarla con $\bar{\lambda} = \mu/(1 - n)$.

Domande guida

- gli eventi si distribuiscono uniformemente nel tempo o si vedono chiaramente grappoli?
- l'intensita' media simulata e' vicina al valore teorico?
- come cambia il clustering al variare di n ?

Output richiesto

- codice sorgente;
- visualizzazione degli eventi e dell'intensità $\lambda^*(t)$;
- confronto tra intensità media simulata e teorica;
- commento visivo sul clustering.

11.2 Laboratorio 2 - Branching ratio e stabilità

Obiettivo

Studiare la transizione tra regime stazionario e regime esplosivo al variare di n .

Attività

1. fissare $\mu = 0.5$, $\beta = 1.0$ e variare α in modo da coprire $n = 0.3, 0.6, 0.9, 0.95, 1.0$;
2. simulare il processo per $T = 500$ per ogni valore di n ;
3. misurare il numero totale di eventi e confrontare con $\mu T / (1 - n)$ per $n < 1$;
4. osservare cosa accade per $n \geq 1$.

Domande guida

- come cambia il numero di eventi al crescere di n ?
- per n vicino a 1, il numero di eventi è molto variabile tra diverse realizzazioni?
- per $n \geq 1$, il processo esplode sempre o solo a volte?

Output richiesto

- grafici del numero di eventi in funzione di n ;
- confronto con il valore teorico atteso;
- discussione del comportamento critico vicino a $n = 1$.

11.3 Laboratorio 3 - Inferenza e diagnostica

Obiettivo

Stimare i parametri di un processo di Hawkes da una sequenza di eventi osservata e verificare la bontà del modello.

Attività

1. simulare una sequenza di eventi con parametri noti (μ, α, β) ;
2. implementare la log-verosimiglianza e massimizzarla;
3. confrontare i parametri stimati con quelli veri;
4. calcolare i residui trasformati τ_k e verificare che siano distribuiti esponenzialmente.

Domande guida

- la stima di massima verosimiglianza converge ai parametri veri al crescere di T ?
- il QQ-plot dei residui trasformati e' vicino alla retta diagonale?
- quali parametri sono piu' difficili da stimare?

Output richiesto

- tabella dei parametri veri e stimati per diversi valori di T ;
- QQ-plot dei residui trasformati;
- commento sulla diagnostica del modello.

11.4 Laboratorio 4 - Processo multivariato e rete di influenza

Obiettivo

Simulare un processo di Hawkes bivariato e studiare la struttura di eccitazione incrociata.

Attivita'

1. definire due tipi di evento con matrice di influenza 2×2 ;
2. simulare il processo bivariato;
3. stimare la matrice di influenza dai dati simulati;
4. costruire il grafico orientato delle influenze e verificare la struttura di causalita'.

Domande guida

- e' possibile identificare correttamente la matrice di influenza dalla simulazione?
- eventi di tipo 1 eccitano di piu' se stessi o il tipo 2?
- come cambia la struttura dei cluster al variare della matrice di influenza?

Output richiesto

- sequenze temporali per i due tipi di eventi;
- matrice di influenza stimata vs vera;
- grafico delle influenze;
- commento sulla causalita' inferita.

12. Una possibile estensione teorica

12.1 Connessione con i processi di branching

Come accennato, il processo di Hawkes ammette una rappresentazione come processo di Galton-Watson a tempo continuo. Ogni evento genera una progenie di discendenti secondo un processo di Poisson con intensità $\phi(s)$.

La dimensione totale dell'albero di discendenza di un singolo immigrante è una variabile aleatoria la cui distribuzione dipende da n . Per $n < 1$, la dimensione media dell'albero è $1/(1 - n)$. Per $n = 1$, la dimensione media è infinita ma la dimensione rimane finita quasi certamente. Per $n > 1$, l'albero è infinito con probabilità positiva.

Questa connessione rende il processo di Hawkes un ponte naturale tra i processi di Poisson, i processi di branching e le catene di Markov, tre dei modelli fondamentali del corso.

12.2 Connessione con i modelli SIR

Il parametro R_0 dei modelli epidemici corrisponde esattamente al branching ratio n . Un'epidemia è una forma di processo di Hawkes in cui ogni infetto genera in media R_0 nuovi infetti. La differenza principale è che nei modelli SIR la popolazione è finita e la suscettibilità diminuisce nel tempo (gli infetti diventano immuni), mentre nel processo di Hawkes la popolazione è infinita e la suscettibilità è costante.

Questa connessione è molto utile didatticamente perché permette di leggere i risultati di un progetto alla luce dell'altro.

13. Perché questo è un buon case study per il corso

Questo progetto è particolarmente adatto a un corso di metodi computazionali per modelli stocastici per almeno cinque ragioni.

Primo, introduce un ingrediente concettuale nuovo rispetto a tutti gli altri modelli del corso: la **dipendenza dalla storia passata**. Nei modelli di Vicsek, March, Cournot, l'interazione è tra agenti presenti nello stesso istante. Nel processo di Hawkes, ogni evento lascia una "traccia" che influenza il futuro.

Secondo, il modello è applicabile a dati reali molto più direttamente di molti altri modelli del corso. Sequenze di terremoti, transazioni finanziarie, post sui social media sono disponibili pubblicamente e si prestano immediatamente all'analisi con un processo di Hawkes.

Terzo, la connessione con l'inferenza e' molto naturale e operativa: la log-verosimiglianza si calcola in forma chiusa e la diagnostica dei residui e' concettualmente limpida.

Quarto, il modello multivariato introduce il concetto di causalita' di Granger in modo molto concreto: un tipo di evento ne "causa" un altro se la matrice di influenza ha un elemento non nullo nella posizione corrispondente.

Quinto, le connessioni con i processi di branching, i modelli SIR e la teoria delle code rendono il progetto un ottimo strumento di sintesi trasversale del corso.

14. Conclusione

Il processo di Hawkes mostra come la dipendenza dalla storia passata trasformi un semplice processo di Poisson in un modello capace di descrivere clustering, cascate, contagio e propagazione.

Il parametro fondamentale e' il branching ratio n : per $n < 1$ il sistema e' stabile e stazionario, per $n \rightarrow 1$ il sistema e' critico e produce cascate di grandi dimensioni, per $n > 1$ il sistema esplode.

Dal punto di vista metodologico, il progetto combina in modo naturale:

- definizione rigorosa di un processo puntuale tramite l'intensita' condizionata;
- simulazione efficiente con il metodo del thinning;
- analisi delle proprieta' stazionarie;
- inferenza statistica tramite massima verosimiglianza;
- diagnostica con i residui trasformati;
- estensione multivariata e analisi delle reti di influenza.

Il messaggio concettuale piu' importante e' che gli eventi raramente sono indipendenti: il passato conta, e la struttura di questa dipendenza ha conseguenze macroscopiche misurabili.

15. Bibliografia minima

1. Hawkes, A. G. (1971). Spectra of Some Self-Exciting and Mutually Exciting Point Processes. *Biometrika*, 58(1), 83-90.
2. Ogata, Y. (1981). On Lewis' Simulation Method for Point Processes. *IEEE Transactions on Information Theory*, 27(1), 23-31.
3. Ogata, Y. (1988). Statistical Models for Earthquake Occurrences and Residual Analysis for Point Processes. *Journal of the American Statistical Association*, 83(401), 9-27.
4. Bacry, E., Mastromatteo, I., and Muzy, J.-F. (2015). Hawkes Processes in Finance. *Market Microstructure and Liquidity*, 1(1).

5. Laub, P. J., Lee, Y., and Taimre, T. (2021). The Elements of Hawkes Processes. Springer.
-

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python il processo di Hawkes univariato e bivariato, la simulazione con thinning, la log-verosimiglianza e la diagnostica.

Il codice e' volutamente elementare:

- poche librerie;
- funzioni corte;
- passaggi espliciti;
- nomi leggibili.

A.1 Librerie minime

```
import random
import math
import statistics
import matplotlib.pyplot as plt
```

Non e' necessario usare `numpy` in una prima implementazione. Per la minimizzazione della log-verosimiglianza si usa `scipy.optimize.minimize`, che e' parte della distribuzione standard di SciPy.

A.2 Intensita' condizionata con kernel esponenziale

Con il kernel esponenziale $\phi(s) = \alpha e^{-\beta s}$, l'intensita' si aggiorna in modo ricorsivo. Definiamo prima una funzione che calcola l'intensita' in un tempo t data la storia degli eventi:

```
def hawkes_intensity(t, events, mu, alpha, beta):
    intensity = mu

    for t_k in events:
        if t_k < t:
            intensity += alpha * math.exp(-beta * (t - t_k))

    return intensity
```

Questa funzione e' $O(n)$ nel numero di eventi passati. E' utile per calcolare l'intensita' in un punto preciso, ma per la simulazione useremo un aggiornamento ricorsivo piu' efficiente.

A.3 Simulazione con il metodo di thinning (algoritmo di Ogata)

```
def simulate_hawkes(mu, alpha, beta, T, max_events=100000):
    events = []
    t = 0.0
    intensity = mu

    while t < T and len(events) < max_events:
        # upper bound sull'intensita' fino al prossimo evento
        # con kernel esponenziale, l'intensita' e' decrescente
        # quindi Lambda = intensita' corrente e' un upper bound valido
        Lambda = intensity

        if Lambda <= 0.0:
            break

        # genera il candidato
        dt = random.expovariate(Lambda)
        t_candidate = t + dt

        if t_candidate > T:
            break

        # calcola l'intensita' effettiva al tempo candidato
        # l'intensita' e' decaduta esponenzialmente da t a t_candidate
        intensity_at_candidate = mu + (intensity - mu) * math.exp(-beta * dt)

        # accetta o rifiuta
        u = random.random()
        if u <= intensity_at_candidate / Lambda:
            # evento accettato
            events.append(t_candidate)
            # aggiorna l'intensita': aggiungi alpha e aggiorna il tempo
            intensity = intensity_at_candidate + alpha
            t = t_candidate
        else:
            # evento rifiutato: aggiorna solo il tempo e l'intensita' decaduta
            intensity = intensity_at_candidate
            t = t_candidate

    return events
```

Nota: l'aggiornamento `intensity = intensity_at_candidate + alpha` sfrutta il fatto che con il kernel esponenziale l'intensita' (al netto del background μ) decade esponenzialmente, e ogni nuovo evento aggiunge α all'intensita' corrente.

Questo rende l'algoritmo $O(n)$ invece di $O(n^2)$.

Esempio:

```
events = simulate_hawkes(mu=0.5, alpha=0.8, beta=1.0, T=200.0)
print(f"Numero di eventi simulati: {len(events)}")

# intensita' media teorica
n = 0.8 / 1.0
lambda_bar_theory = 0.5 / (1.0 - n)
lambda_bar_empirical = len(events) / 200.0
print(f"Intensita' media teorica: {lambda_bar_theory:.4f}")
print(f"Intensita' media empirica: {lambda_bar_empirical:.4f}")
```

A.4 Visualizzazione degli eventi e dell'intensita'

```
def plot_hawkes_events(events, mu, alpha, beta, T, num_points=1000):
    # traiettoria dell'intensita'
    times = [T * k / num_points for k in range(num_points + 1)]
    intensities = [hawkes_intensity(t, events, mu, alpha, beta) for t in times]

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6), sharex=True)

    # pannello superiore: eventi come barre verticali
    for t_k in events:
        ax1.axvline(x=t_k, color="black", alpha=0.5, linewidth=0.7)
    ax1.set_ylabel("eventi")
    ax1.set_yticks([])
    ax1.set_title("Processo di Hawkes: eventi e intensita' condizionata")

    # pannello inferiore: intensita'
    ax2.plot(times, intensities, color="steelblue")
    ax2.axhline(y=mu / (1.0 - alpha / beta), color="red",
                linestyle="--", label="intensita' media teorica")
    ax2.set_xlabel("tempo")
    ax2.set_ylabel("lambda*(t)")
    ax2.legend()

    plt.tight_layout()
    plt.show()
```

Esempio:

```
plot_hawkes_events(events, mu=0.5, alpha=0.8, beta=1.0, T=200.0)
```

A.5 Log-verosimiglianza per il kernel esponenziale

Per una sequenza di n eventi $\{t_1, \dots, t_n\}$ osservata su $[0, T]$, la log-verosimiglianza è:

$$\log L = \sum_{k=1}^n \log \lambda^*(t_k) - \int_0^T \lambda^*(t) dt.$$

Per il kernel esponenziale, l'integrale si calcola in forma chiusa:

$$\int_0^T \lambda^*(t) dt = \mu T + \frac{\alpha}{\beta} \sum_{k=1}^n \left(1 - e^{-\beta(T-t_k)}\right).$$

```
def hawkes_log_likelihood(events, mu, alpha, beta, T):
    if mu <= 0 or alpha <= 0 or beta <= 0:
        return -float("inf")

    n = len(events)

    if n == 0:
        return -mu * T

    # calcola le intensita' agli eventi in modo ricorsivo
    log_sum = 0.0
    intensity = mu
    t_prev = 0.0

    for t_k in events:
        # decade dall'ultimo evento
        dt = t_k - t_prev
        intensity = mu + (intensity - mu) * math.exp(-beta * dt)

        if intensity <= 0.0:
            return -float("inf")

        log_sum += math.log(intensity)

        # aggiorna dopo l'evento
        intensity += alpha
        t_prev = t_k

    # termine integrale
    integral = mu * T
    for t_k in events:
        integral += (alpha / beta) * (1.0 - math.exp(-beta * (T - t_k)))
```

```
return log_sum - integral
```

A.6 Stima di massima verosimiglianza

Per massimizzare la log-verosimiglianza usiamo `scipy.optimize.minimize` con il segno cambiato (minimizziamo la negativa):

```
def fit_hawkes(events, T, mu_init=1.0, alpha_init=0.5, beta_init=1.0):
    from scipy.optimize import minimize

    def neg_log_likelihood(params):
        mu, alpha, beta = params
        return -hawkes_log_likelihood(events, mu, alpha, beta, T)

    result = minimize(
        neg_log_likelihood,
        x0=[mu_init, alpha_init, beta_init],
        method="L-BFGS-B",
        bounds=[(1e-6, None), (1e-6, None), (1e-6, None)]
    )

    mu_hat, alpha_hat, beta_hat = result.x

    return {
        "mu": mu_hat,
        "alpha": alpha_hat,
        "beta": beta_hat,
        "n": alpha_hat / beta_hat,
        "log_likelihood": -result.fun,
        "converged": result.success
    }
```

Esempio:

```
true_params = {"mu": 0.5, "alpha": 0.8, "beta": 1.0}
events = simulate_hawkes(T=500.0, **true_params)

fit = fit_hawkes(events, T=500.0)

print("Parametri veri: mu={mu}, alpha={alpha}, beta={beta}".format(**true_params))
print("Parametri stimati: mu={mu:.4f}, alpha={alpha:.4f}, beta={beta:.4f}".format(**fit))
print("Branching ratio stimato:", round(fit["n"], 4))
```

A.7 Diagnostica: residui trasformati

Il teorema di rescaling afferma che se il modello e' corretto, i tempi trasformati

$$\tau_k = \int_0^{t_k} \lambda^*(s) ds$$

formano un processo di Poisson omogeneo con tasso 1.

```
def transformed_times(events, mu, alpha, beta):
    taus = []
    cumulative = 0.0
    t_prev = 0.0
    intensity_prev = mu

    for t_k in events:
        dt = t_k - t_prev

        # integrale dell'intensita' da t_prev a t_k
        # con kernel esponenziale: integral = mu*dt + (intensity_prev - mu)/beta * (1 - exp
        integral = mu * dt + (intensity_prev - mu) / beta * (1.0 - math.exp(-beta * dt))
        cumulative += integral

        taus.append(cumulative)

        # aggiorna
        intensity_prev = mu + (intensity_prev - mu) * math.exp(-beta * dt) + alpha
        t_prev = t_k

    return taus
```

I tempi inter-evento trasformati devono essere esponenziali con media 1:

```
def plot_residual_diagnostics(events, mu, alpha, beta):
    taus = transformed_times(events, mu, alpha, beta)

    # inter-event times trasformati
    inter_taus = [taus[k] - taus[k-1] for k in range(1, len(taus))]

    # QQ-plot: confronto con esponenziale(1)
    n = len(inter_taus)
    inter_taus_sorted = sorted(inter_taus)
    theoretical_quantiles = [-math.log(1.0 - (k + 0.5) / n) for k in range(n)]

    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.plot(theoretical_quantiles, inter_taus_sorted, ".", markersize=3)
    plt.plot([0, max(theoretical_quantiles)],
             [0, max(theoretical_quantiles)], "r--")
```

```

plt.xlabel("quantili teorici Exp(1)")
plt.ylabel("quantili empirici")
plt.title("QQ-plot dei residui trasformati")

plt.subplot(1, 2, 2)
plt.hist(inter_taus, bins=30, density=True)
x_vals = [0.1 * k for k in range(1, 50)]
y_vals = [math.exp(-x) for x in x_vals]
plt.plot(x_vals, y_vals, "r-", label="Exp(1) teorica")
plt.xlabel("inter-event time trasformato")
plt.ylabel("densita'")
plt.title("Distribuzione dei residui")
plt.legend()

plt.tight_layout()
plt.show()

```

Esempio:

```

fit = fit_hawkes(events, T=500.0)
plot_residual_diagnostics(events, mu=fit["mu"], alpha=fit["alpha"], beta=fit["beta"])

```

A.8 Simulazione del processo bivariato

Nel processo bivariato ci sono due tipi di evento. L'intensita' di ciascun tipo dipende dalla storia di entrambi i tipi:

$$\lambda_1^*(t) = \mu_1 + \alpha_{11} \sum_{t_k^{(1)} < t} e^{-\beta_{11}(t-t_k^{(1)})} + \alpha_{12} \sum_{t_k^{(2)} < t} e^{-\beta_{12}(t-t_k^{(2)})}$$

```

def simulate_bivariate_hawkes(mu, alpha, beta, T, max_events=200000):
    # mu: lista [mu1, mu2]
    # alpha: matrice 2x2, alpha[m][j] = eccitazione di tipo m da tipo j
    # beta: matrice 2x2, beta[m][j] = tasso decadimento
    # Restituisce due liste di eventi: events[0] e events[1]

    events = [[], []]
    t = 0.0

    # intensita' correnti per i due tipi
    intensities = [mu[0], mu[1]]

    while t < T and sum(len(e) for e in events) < max_events:
        # upper bound: somma delle intensita' dei due tipi
        Lambda = sum(intensities)

        if Lambda <= 0.0:

```

```

        break

dt = random.expovariate(Lambda)
t_candidate = t + dt

if t_candidate > T:
    break

# decadi le intensita' fino a t_candidate
intensities_at_candidate = []
for m in range(2):
    decay = (intensities[m] - mu[m]) * math.exp(
        -min(beta[m][0], beta[m][1]) * dt
    )
    intensities_at_candidate.append(mu[m] + decay)

# in realta' i due tipi hanno tassi di decadimento diversi
# calcola piu' precisamente
intensity1 = mu[0]
for t_k in events[0]:
    if t_k < t_candidate:
        intensity1 += alpha[0][0] * math.exp(-beta[0][0] * (t_candidate - t_k))
for t_k in events[1]:
    if t_k < t_candidate:
        intensity1 += alpha[0][1] * math.exp(-beta[0][1] * (t_candidate - t_k))

intensity2 = mu[1]
for t_k in events[0]:
    if t_k < t_candidate:
        intensity2 += alpha[1][0] * math.exp(-beta[1][0] * (t_candidate - t_k))
for t_k in events[1]:
    if t_k < t_candidate:
        intensity2 += alpha[1][1] * math.exp(-beta[1][1] * (t_candidate - t_k))

lambda_total = intensity1 + intensity2

u = random.random()
if u * Lambda <= lambda_total:
    # decide il tipo dell'evento
    u2 = random.random()
    if u2 * lambda_total <= intensity1:
        event_type = 0
    else:
        event_type = 1

    events[event_type].append(t_candidate)

```

```

        t = t_candidate

    return events

```

A.9 Visualizzazione del processo bivariato

```

def plot_bivariate_events(events, T, title="Processo di Hawkes bivariato"):
    fig, axes = plt.subplots(2, 1, figsize=(12, 4), sharex=True)

    for m in range(2):
        for t_k in events[m]:
            axes[m].axvline(x=t_k, color="steelblue" if m == 0 else "darkorange",
                           alpha=0.6, linewidth=0.8)
        axes[m].set_ylabel(f"tipo {m + 1}")
        axes[m].set_yticks([])

    axes[1].set_xlabel("tempo")
    axes[0].set_title(title)
    plt.tight_layout()
    plt.show()

```

A.10 Conteggio degli eventi in finestre temporali

Una rappresentazione alternativa degli eventi e' il conteggio in finestre di ampiezza fissa, utile per confrontare visivamente il clustering:

```

def count_events_in_windows(events, T, window_size):
    num_windows = int(T / window_size)
    counts = [0] * num_windows

    for t_k in events:
        window_idx = int(t_k / window_size)
        if window_idx < num_windows:
            counts[window_idx] += 1

    return counts

def plot_event_counts(events, T, window_size=1.0, title="Conteggio degli eventi"):
    counts = count_events_in_windows(events, T, window_size)
    times = [window_size * k for k in range(len(counts))]

    plt.bar(times, counts, width=window_size * 0.9, align="edge")
    plt.xlabel("tempo")
    plt.ylabel("numero di eventi")

```

```
plt.title(title)
plt.show()
```

A.11 Stima del branching ratio dai dati

Una stima molto semplice (non la piu' efficiente, ma didatticamente utile) del branching ratio si ottiene come rapporto tra il numero di eventi generati dal processo di background e il numero totale:

```
def estimate_branching_ratio_simple(events, mu_hat, T):
    n_total = len(events)
    n_background_expected = mu_hat * T

    if n_total == 0:
        return 0.0

    n_hat = 1.0 - n_background_expected / n_total
    return max(0.0, min(n_hat, 0.999))
```

Questa stima e' rozza ma intuitiva: la frazione di eventi non spiegata dal background e' quella generata dall'auto-eccitazione.

A.12 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo in questo ordine:

1. import delle librerie;
2. funzioni di base:
 - `hawkes_intensity`
3. simulazione:
 - `simulate_hawkes`
 - `simulate_bivariate_hawkes`
4. visualizzazione:
 - `plot_hawkes_events`
 - `plot_bivariate_events`
 - `plot_event_counts`
5. inferenza:
 - `hawkes_log_likelihood`
 - `fit_hawkes`
6. diagnostica:
 - `transformed_times`
 - `plot_residual_diagnostics`
7. utilita':
 - `count_events_in_windows`
 - `estimate_branching_ratio_simple`
8. blocco finale con esempi.

Per esempio:


```

if __name__ == "__main__":
    mu = 0.5
    alpha = 0.8
    beta = 1.0
    T = 300.0

    print("=== Simulazione ===")
    events = simulate_hawkes(mu=mu, alpha=alpha, beta=beta, T=T)
    n_theory = alpha / beta
    lambda_bar_theory = mu / (1.0 - n_theory)
    print(f"Numero eventi: {len(events)}")
    print(f"Intensita' media teorica: {lambda_bar_theory:.4f}")
    print(f"Intensita' media empirica: {len(events)/T:.4f}")

    plot_hawkes_events(events, mu=mu, alpha=alpha, beta=beta, T=T)

    print("\n=== Inferenza ===")
    fit = fit_hawkes(events, T=T, mu_init=0.3, alpha_init=0.5, beta_init=0.8)
    print(f"Veri:      mu={mu}, alpha={alpha}, beta={beta}, n={n_theory:.4f}")
    print(f"Stimati: mu={fit['mu']:.4f}, alpha={fit['alpha']:.4f}, "
          f"beta={fit['beta']:.4f}, n={fit['n']:.4f}")

    print("\n=== Diagnostica ===")
    plot_residual_diagnostics(events, mu=fit["mu"],
                              alpha=fit["alpha"], beta=fit["beta"])

```

A.13 Perché questa appendice è utile

Questa appendice ha tre funzioni didattiche principali.

Primo, l'algoritmo di thinning è implementato in modo molto esplicito, rendendo visibile ogni passo: generazione del candidato, calcolo dell'intensità effettiva, accettazione o rifiuto. Non c'è nulla di nascosto.

Secondo, la log-verosimiglianza è implementata in forma ricorsiva efficiente, mostrando come la struttura del kernel esponenziale permetta di evitare il calcolo $O(n^2)$ che sembrerebbe necessario.

Terzo, la diagnostica dei residui trasformati è immediatamente eseguibile e fornisce un test visivo molto chiaro sulla bontà del modello: se il QQ-plot è vicino alla diagonale, il modello è adeguato; se se ne discosta, bisogna riconsiderare la specificazione.

A.14 Conclusione dell'appendice

La struttura proposta è volutamente semplice. Chi conosce Python può implementarla quasi direttamente; chi usa altri linguaggi può leggerla come

pseudocodice molto vicino a una traduzione operativa.

Il messaggio metodologico e' che il processo di Hawkes, nonostante introduca la dipendenza dalla storia passata, rimane computazionalmente molto trattabile: la simulazione e' $O(n)$, la log-verosimiglianza e' $O(n)$, e la diagnostica e' operativa in poche righe di codice.